

LLM Fine-Tuning

LoRA, QLoRA, and Parameter-Efficient Methods

Hu et al. LoRA (2021) | Dettmers QLoRA (2023) | HuggingFace TRL

Overview

Fine-tuning adapts a pre-trained LLM to specific tasks, domains, or styles using a curated dataset. This document covers the major fine-tuning approaches, tools, and best practices for production LLM systems.

1. When to Fine-tune vs. Prompt Engineering

Approach	When to Use	Cost	Flexibility
Prompt engineering	Task is achievable in-context; few-shot works	Low	High
RAG	Knowledge needs to be updatable; source attribution required	Medium	High
Fine-tuning	Consistent format/style; domain jargon; insufficient context window	High	Low
Pre-training	Entirely new domain (medical, legal, code); data > 10GB tokens	Very High	Low

2. LoRA and Parameter-Efficient Fine-tuning

Full fine-tuning of a 7B model requires ~28GB GPU memory (fp16). Parameter-efficient methods dramatically reduce this:

- LoRA (Low-Rank Adaptation):** Hu et al. (2021). Adds trainable low-rank matrices to attention weights. $\Delta W = A*B$ where rank $r \ll d_{\text{model}}$ (e.g., $r=8$ vs $d=4096$). Trains <1% of parameters; full-rank performance. Standard for LLM fine-tuning.
- QLoRA (Quantized LoRA):** Dettmers et al. (2023). 4-bit quantization + LoRA. Fine-tune 65B model on single 48GB GPU. Uses NF4 (Normal Float 4) quantization with double quantization.
- IA3:** Few-Shot Parameter-Efficient Fine-Tuning. Scales activations with learned vectors. Even fewer parameters than LoRA.
- Prefix Tuning / P-Tuning:** Learn soft prompts prepended to each layer. No change to model weights. Lower performance than LoRA but simpler.

3. Fine-tuning with HuggingFace TRL + LoRA

```

from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import LoraConfig, get_peft_model, TaskType
from trl import SFTTrainer, SFTConfig

model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.2-3B-Instruct",
    torch_dtype="bfloat16",
    device_map="auto",
    load_in_4bit=True # QLoRA
)

lora_config = LoraConfig(
    r=16, # rank
    lora_alpha=32, # scaling
    target_modules=["q_proj", "v_proj", "k_proj", "o_proj"],
    lora_dropout=0.05,
    task_type=TaskType.CAUSAL_LM
)
model = get_peft_model(model, lora_config)
model.print_trainable_parameters() # ~0.5% of params

trainer = SFTTrainer(
    model=model,
    train_dataset=dataset,
    tokenizer=tokenizer,
    args=SFTConfig(
        output_dir="./llama-finetuned",
        num_train_epochs=3,
        per_device_train_batch_size=4,
        gradient_accumulation_steps=4,
        learning_rate=2e-4,
        warmup_ratio=0.1,
        lr_scheduler_type="cosine",
    ),
)
trainer.train()
model.save_pretrained("./my-fine-tuned-model")

```

4. Dataset Preparation

For instruction fine-tuning, format data as:

```

{"messages": [{"role": "system", "content": "..."}, {"role": "user", "content": "..."}, {"role": "assistant", "content": "..."}]}

```

- Quality > quantity: 1000 high-quality examples > 100k noisy ones (LIMA, 2023)
- Deduplicate rigorously — near-duplicate training samples cause overfitting
- Include ~5–10% general instruction data to prevent catastrophic forgetting

- Validate with held-out eval set; target 10–15% of training size